

Railway Ticket Booking System Using Linked List and Queue in C

Abstract

The objective of this project is to develop a console-based Railway Ticket Booking System using the C programming language, focusing on the use of fundamental data structures such as linked lists and queues. The system allows users to book and cancel tickets for multiple trains, manage a waitlist, and ensure smooth allocation of available seats. The application demonstrates real-life simulation of queue handling, dynamic memory allocation, and pointer-based data management in C. This project emphasizes practical application of Data Structures and Algorithms (DSA) in a structured and modular approach.

Problem Description

The **Railway Ticket Booking System** code aims to efficiently manage ticket reservations, cancellations, and waitlists in a simplified console-based environment. The system simulates the real-world process of booking train tickets with limited seat capacity and an automatic waitlist mechanism.

Passengers can select a train from the available list and proceed to book tickets. If seats are available, they are assigned a **confirmed seat number**. Once all seats are booked, any further booking requests are added to a **waitlist queue** on a **First-In-First-Out (FIFO)** basis. In the event of a ticket cancellation, the system automatically assigns the freed seat to the **first passenger in the waitlist**, promoting them to the confirmed list.

The problem aims to simulate:

- Train selection by the user
- Booking tickets until all seats are filled
- Adding passengers to the waitlist if confirmed seats are unavailable
- Promoting waitlisted passengers when cancellations occur

Implementation Details

The system is implemented in the C language with the following key features:

- **Menu-driven interface** that supports booking, cancellation, and display of passenger details.
- **Multiple train support:** Each train has its own confirmed passenger list and waitlist.
- **Dynamic memory management:** All data is stored in dynamically allocated structures.
- **Waitlist handling:** When a confirmed passenger cancels, the first person in the waitlist is moved to the confirmed list.

The program flow includes:

1. Initializing train data (name, ID, route, seat capacity)
2. Displaying a menu for user interaction
3. Handling input and updating linked list and queue accordingly

Data Structures Used

The project makes use of fundamental data structures in C to effectively manage booking, cancellation, and waitlisting operations.

A singly linked list is used to maintain the list of confirmed passengers for each train. Each node in the linked list represents a passenger and stores details such as name, age, seat number, and booking status. This allows dynamic addition and deletion of passengers without a fixed limit on size, making it suitable for a flexible and changing list of confirmed bookings.

To manage the waitlist, the project uses a queue implemented using a linked list. The queue follows the First-In-First-Out (FIFO) principle, ensuring that passengers are promoted to the confirmed list in the same order they joined the waitlist. Enqueue and dequeue operations help manage the entry and exit of passengers in the waitlist in an efficient manner.

Additionally, an array of structures is used to store information about multiple trains. Each train in the array holds its own confirmed list and waitlist queue, along with attributes such as train name, train number, route, and seat capacity.

These data structures work together ensuring accurate and efficient handling of passenger data across different operations.

Challenges Faced

- **Pointer Management:** Proper handling of dynamic memory and pointers was challenging, especially while linking or unlinking passenger nodes during cancellation or queue shifting.
- **Multi-train Support:** Managing multiple sets of confirmed and waitlisted passengers required clean separation of data per train.
- **Queue Management:** Implementing enqueue and dequeue logic without bugs and ensuring proper linking during waitlist movement needed extra care.

Conclusion and Learnings

This project helped reinforce key programming concepts such as:

- **Linked lists and queue implementation**
- **Modular programming in C**
- **Memory allocation and pointer handling**
- **Real-world application modeling using data structures**

It also enhanced problem-solving skills by requiring us to simulate a real-world system using DSA principles.

OUTPUT

```
Railway Ticket Booking System
1. Book Ticket
2. Cancel Ticket
3. Display Bookings
4. Exit
Enter choice: 1

Available Trains:
1. Express A (Kolkata to Shimla)
2. Express B (Pune to Kolkata)
3. Express C (Hyderabad to Delhi)
Select a train (1-3): 1
Enter Passenger Name: Shreya
Enter Age: 18

Ticket booked on Express A. Shreya got seat 1.

Railway Ticket Booking System
1. Book Ticket
2. Cancel Ticket
3. Display Bookings
4. Exit
Enter choice: 1

Available Trains:
1. Express A (Kolkata to Shimla)
2. Express B (Pune to Kolkata)
3. Express C (Hyderabad to Delhi)
Select a train (1-3): 1
Enter Passenger Name: Sara
Enter Age: 13

Ticket booked on Express A. Sara got seat 2.

Railway Ticket Booking System
1. Book Ticket
2. Cancel Ticket
3. Display Bookings
4. Exit
Enter choice: 1

Available Trains:
1. Express A (Kolkata to Shimla)
2. Express B (Pune to Kolkata)
3. Express C (Hyderabad to Delhi)
Select a train (1-3): 2
Enter Passenger Name: Riya
Enter Age: 40

Ticket booked on Express B. Riya got seat 1.
```

```
Railway Ticket Booking System
1. Book Ticket
2. Cancel Ticket
3. Display Bookings
4. Exit
Enter choice: 3

Express A (Kolkata to Shimla):

Confirmed Passengers:
Shreya (Age: 18) - Seat 1 [CONFIRMED]
Sara (Age: 13) - Seat 2 [CONFIRMED]

Waitlisted Passengers:
No passengers in waitlist.

Express B (Pune to Kolkata):

Confirmed Passengers:
Riya (Age: 40) - Seat 1 [CONFIRMED]

Waitlisted Passengers:
No passengers in waitlist.

Express C (Hyderabad to Delhi):

Confirmed Passengers:
No confirmed bookings.

Waitlisted Passengers:
No passengers in waitlist.
```